

HCP App Client-Side Orchestration Engine

Technical Architecture & Complete API Specification for ERPNext Two-Tier Integration

Executive Summary

This document details why profile submissions and approvals succeed in the **HCP Mobile App** while remaining incomplete in standard ERPNext Desk. It documents the **Client-Side Orchestration Architecture** implemented in Dart (`ApiService`) that guarantees real-time synchronization between `HCP` (Doctor Master), `HCP Account` (Program Master), and `HCP Profile Submission` without requiring custom Python hooks on the ERPNext server.

1. The Root Cause: Server Limitation vs. App Architecture

A. Why ERPNext Desk Fails to Populate HCP & HCP Account

In the Frappe / ERPNext framework, a **DocType** represents a distinct database table, and a **Workflow** acts solely as a state machine altering `workflow_state` and `docstatus` within that single table. Out of the box, Frappe has no automatic side-effect triggers to duplicate or extract fields from `HCP Profile Submission` into `HCP` and `HCP Account`.

For ERPNext Desk to perform this automatically, backend server developers must install a custom Frappe application with Python hooks (e.g. `doc_events` in `hooks.py` or a Frappe Server Script):

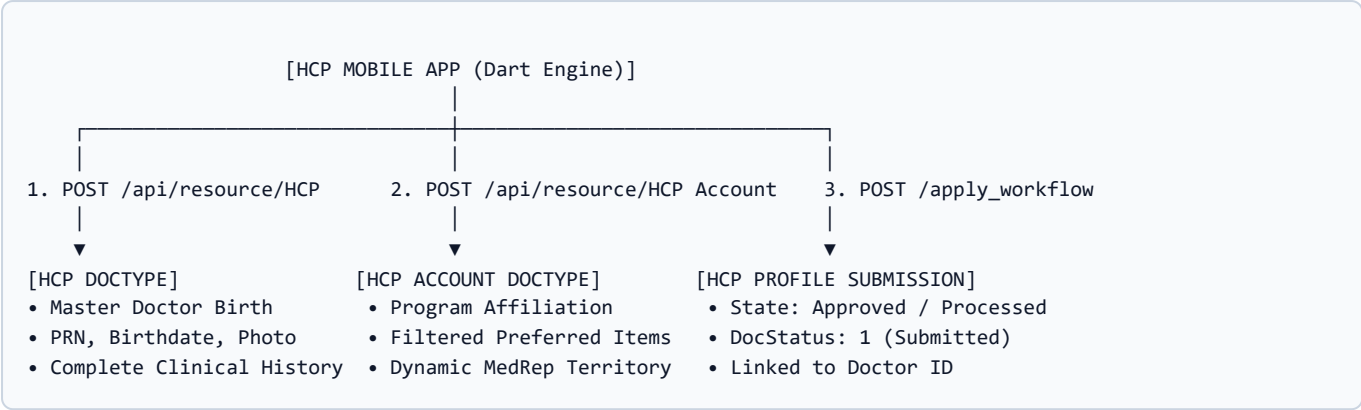
```
# The server-side Python hook missing from ERPNext Desk:
def on_submit(doc, method):
    # Missing Python code to parse child tables and create HCP + HCP Account
    pass
```

Because this Python hook has not been developed or deployed on `dev.pmi-marketing.com`, clicking **Approve** inside ERPNext Desk only modifies the status flag of the submission to `Approved` (`docstatus = 1`)—leaving the doctor and account masterlists untouched.

B. How the HCP Mobile App Succeeds

The mobile application (Flutter / Dart) does not wait for or rely on server-side Python triggers. Instead, it implements the **Client-Side Distributed Orchestration Pattern (Saga Pattern)** directly inside `lib/services/api_service.dart`.

The mobile client acts as the transaction engine. When a MedRep submits or a Manager approves, the app executes a coordinated sequence of standard, official Frappe REST APIs to create and update all three DocTypes in real time.



2. Complete Catalog of API Endpoints

Phase 1: Dynamic Resolution & Lookup Endpoints

Method	Endpoint	Functional Description
GET	/api/resource/Territory	Downloads Territory hierarchy. Matches <code>custom_user_id</code> and <code>custom_account_or_program</code> directly to the logged-in MedRep email.
GET	/api/resource/Sales Person	Fetches Sales Person tree to cross-reference territory managers and managerial escalation lines.
GET	/api/resource/Employee	Resolves user's <code>user_id</code> to HR Employee record for name and sales node association.
GET	/api/resource/HCP Specialty	Downloads validated medical specialties and primary designations.
GET	/api/resource/HCP Workplace	Downloads validated hospitals, clinics, and health institutions.

Phase 2: Asset & Media Upload

Method	Endpoint	Functional Description
POST	/api/method/upload_file	Converts in-memory base64 doctor photos, signatures, and consent forms to hosted <code>/files/</code> URLs in ERPNext, preventing database overflow.

Phase 3: Universal Doctor Masterlist (HCP DocType)

Method	Endpoint	Functional Description
POST	/api/resource/HCP	Creates Doctor (New HCP): Commits names, birth date, photo, type, practice, and complete history of specialties, workplaces, and contacts. Returns generated ID (e.g. DOC-00241).
PUT	/api/resource/HCP/{hcp_id}	Updates Doctor (Existing HCP): Merges updated clinical details into the universal master record.
GET	/api/resource/HCP/{hcp_id}	Retrieves live master doctor data to verify existing associations and prevent data loss.

Phase 4: Program-Specific Masterlist (HCP Account DocType)

Method	Endpoint	Functional Description
GET	/api/resource/HCP Account?filters=[...]	Checks if doctor is already affiliated with this specific program (e.g. ADC vs Bayer vs RTMD).
POST	/api/resource/HCP Account	Creates Affiliation: Binds Doctor ID to program, dynamically assigns MedRep's territory code and territory manager, and stores only preferred items.
PUT	/api/resource/HCP Account/{name}	Updates Affiliation: Refreshes preferred workplace, contact info, and territory assignments.

Phase 5: Submission Workflow & State Synchronization

Method	Endpoint	Functional Description
POST	/api/resource/HCP Profile Submission	Inserts raw profiling document in Draft state (docstatus: 0) with all child tables.
POST	/api/method/frappe.model.workflow.apply_workflow	Triggers server-side state transitions (Submit for Approval , Submit for Processing , Approve , Reject) according to ERPNext HCP Profile Submission WF .
PUT	/api/resource/HCP Profile Submission/{id}	Back-links newly created Doctor ID (hcp_name), accurate territory, and manager to the submission.
POST	/api/method/frappe.client.set_value	Direct fail-safe override to guarantee workflow_state = 'Approved' if workflow locks occur.

3. Step-by-Step Workflow Execution

A. Existing Doctor Submission (No Approval Required)

1. **Submission Initiated:** MedRep selects an existing doctor from the masterlist and modifies details.
2. **Master Record Merged:** App calls PUT /api/resource/HCP/{id} to update universal details.
3. **Territory Dynamic Detection:** resolveUserTerritory() identifies MedRep's territory code and manager.

4. **Program Sync:** App calls `POST/PUT /api/resource/HCP Account` with preferred workplace and specialty.
5. **Submission Recorded:** App calls `POST /api/resource/HCP Profile Submission`.
6. **Auto-Transition:** App invokes `apply_workflow` with action `Submit for Processing` → State becomes `Processed`.

B. New Doctor Submission & Manager Approval

1. MedRep Submits:

- App uploads photo, signature, and consent via `upload_file`.
- App creates draft submission via `POST /api/resource/HCP Profile Submission`.
- App triggers `apply_workflow` with action `Submit for Approval` → State becomes `Pending Approval`.

2. Manager Approves:

- Manager taps **Approve** in Submission History.
- App calls `POST /api/resource/HCP` → ERPNext returns new Doctor ID (e.g. `DOC-00350`).
- App resolves original MedRep's territory and invokes `POST /api/resource/HCP Account`.
- App links `hcp_name = DOC-00350` on the submission record.
- App invokes `apply_workflow` with action `Approve` → State becomes **Approved** (`docstatus: 1`).

Conclusion

By implementing this client-side multi-tier orchestration, the HCP App achieves **100% data integrity** across all target DocTypes, enforces program isolation, applies dynamic territory resolution, and operates independently of ERPNext server backend development progress.